CORNELL TECH | APPLIED MACHINE LEARNING

Human or AI?

Kaggle Competition & Report Template

Report Template: TEMPLATE

Github Repository Template: REPO

Welcome to the competition! Your task is to build a machine learning classifier that predicts whether a passage was written by a human or generated by an AI model. You will use a labeled training set, an additional unlabeled set, and a held-out Kaggle test set. The aim is not just a high leaderboard score: we want you to compare methods, design careful experiments, understand generalization, and explain what your results actually show.

You are encouraged to try several approaches. A well-tested logistic regression or support vector machine is just as legitimate as a neural network. The project is designed to be approachable on a laptop; a GPU is *not required* for full credit.

Team / members	Zhitong Liu (zl2358); Sorathorn Thongpitukthavorn (st2329); Keyan Miao (km2299)
Kaggle team name	team 35
GitHub PR URL	[pending team approval]

Your data

File	Rows	Labels	Purpose
`train.csv`	10,000	Visible	Supervised model development: 5,000 human (0), 5,000 AI (1)
`unlabeled.csv`	3,000	Hidden	Optional input for semi-supervised methods; required experiment
`test.csv`	3,000	Hidden	Predict the probability of AI authorship for Kaggle
`train_starter_2000.csv`	2,000	Visible	Convenient subset of `train.csv` for quick experiments

The starter data is a subset, not 2,000 additional labeled examples. Passages are approximately 75–350 words, with mean length near 210 words.

How the test set works

The test set mixes familiar and unfamiliar writing domains and AI generators, and includes some adversarially modified AI-written passages. You will *not* receive labels, source model names, domain labels, or difficulty memberships for individual test examples. Submit a probability in $[0, 1]$ for each test ID, where 1 means AI-generated. Kaggle evaluates submissions using **area under the ROC curve (AUROC)**. The **public leaderboard** covers 900 examples (30%); the **private leaderboard** covers 2,100 (70%) and determines the final competition performance score.

1. Competition rubric and submission instructions

Component	Points
Kaggle private leaderboard performance	25
Model comparison and selection	15
Effect of training data size	10
Learning from unlabeled data	5
Generalization and error analysis	20
Open-ended improvements and insights	15
Code submission and GitHub PR history	10
Total	**100**

How to use this document. Complete the response fields and tables below; insert plots where requested. Keep your answers concise, specific, and supported by actual experiments. Points appear in brackets beside each question; no separate writing-quality grade is assigned.

How credit is awarded. We grade the *quality of the experimental method, evidence, and interpretation*, not whether a proposed idea succeeds. **Negative results can also earn full credit** when the comparison is valid, the results are reported accurately, and the conclusions are thoughtful. For each question, make clear what you tried, what you measured, and what you learned. Do not invent scores or results.

Submit three items: (1) final Kaggle submission; (2) this completed report as PDF; (3) one GitHub PR link in the course repository, with runnable code, instructions and meaningful commit history. Include the PR link on the title page. Confirm upload locations and deadline time on the course site. The template for the repository is here

Compute and models. Use models and pipelines that you can run and reproduce. Laptop-friendly training is strongly recommended; use of more expensive methods is allowed but not required for full credit. Do not use external LLM APIs to directly label or predict the competition test passages. Use only the supplied labeled dataset for supervised labels; allowed pretrained weights and AI-assisted development should be disclosed.

Validation. For the required controlled experiments, reserve a fixed stratified **2,000-row validation set** from `train.csv`, leaving up to 8,000 rows for training. Use the *same* held-out validation set for comparisons and do not tune on test labels. You may retrain your final Kaggle model on all 10,000 labeled examples after selecting your approach. Where possible, document seeds and avoid source/near-duplicate leakage.

Kaggle grading (25 points). Final performance will use the private AUROC, not leaderboard rank. The intended continuous rule is $25\,\mathrm{clip}((A - 0.50)/(T - 0.50), 0, 1)$, where A is your private AUROC and T is the announced full-credit threshold. A provisional value of $T = 0.90$ is under consideration; the instructors will confirm the fixed threshold before launch. A reference word TF-IDF and logistic regression baseline achieves approximately 0.868 private AUROC.

2. Team information and model summary

Q0.1 [0 points]. List team members and their main contributions.

Team members: Zhitong Liu (zl2358), Sorathorn Thongpitukthavorn (st2329), and Keyan Miao (km2299). Zhitong developed the character TF–IDF baseline, learning-curve and pseudo-labeling experiments, a wording-cue ablation, and the v3 holdout error analysis. Sorathorn (Plum) developed the multi-view models and v3 ensemble. Keyan built a reproducible word TF–IDF baseline script with checks for missing values, duplicates, and labels. He compared logistic regression, a linear SVM, and multinomial naive Bayes, and used cross-validation to tune C.

Q0.2 [0 points]. Summarize your final approach in 3–5 sentences. What is the overall pipeline from raw text to probability?

Our intended final Kaggle submission uses the v3 ensemble. It turns each passage into several views: word and character patterns, a version that masks topic words, and measurements of writing style and formatting. Nine classifiers combine those views into an AI-authorship probability. A second stage adds high-confidence pseudo-labeled passages from the supplied unlabeled and test files, then blends its predictions with the first stage. The final output is a probability for each test ID; hidden test labels are never used for training.

Q0.3 [0 points]. Record your final scores (enter private score only once it is revealed).

Metric	Your score
Validation AUROC	0.999551 (v3 holdout analogue)
Kaggle public AUROC	0.98552 (full v3)
Kaggle private AUROC	[pending]

3. Model comparison and selection (15 points)

Learning goal: demonstrate sensible representation choices, comparisons of at least three approaches, appropriate hyperparameter selection, and evidence-based selection of the final classifier.

Q1.1 [3 points]. How did you preprocess and represent the text? Specify tokenization, normalization, word/character features or embeddings, and any feature extraction choices. Why?

To preprocess and represent the text, we first made two copies of each passage: one with HTML break tags and extra whitespace removed for word and character patterns, and another retaining layout cues.

TF–IDF gives more weight to patterns that are distinctive across passages. V3 uses groups of one to three words, short 3 to 5 character strings within and across words, and a topic-masked view that keeps grammar while hiding most content words.

We also measured writing style (including vocabulary variety, repetition, sentence length, punctuation, and scores from a small trigram language model). These views can complement each other when the test topic changes. We kept layout cues separate because double newlines appear in 66% of AI training passages but no human ones (allbeit an unreliable shortcut). A separate character-only baseline used only 3 to 5 character TF–IDF patterns; its vocabulary was learned from training rows alone.

Q1.2 [5 points]. Which **three or more** modeling approaches did you try? Keep the labeled validation partition fixed across models.

Model	Text representation	Train AUROC	Val. AUROC	Train time
A: word logistic	1 and 2 word patterns	0.9932	0.9367	6.2 s
A2: Naive Bayes	1 and 2 word patterns	0.9790	0.8966	0.02 s
B: logistic	Word & character & 72 style measures	1.0000	0.9743	2.2 s
C: linear SVM	Same features as B	1.0000	0.9766	8.2 s
D: LightGBM	Style measures and compressed word patterns	1.0000	0.9822	0.6 s
E: blend of B/C/D	Combined word, character, and style views	1.0000	0.9837	11.0 s
F: char logistic	Character patterns only	0.999897	0.968976	8.4 s

All rows use the same stratified 8000/2000 split (seed 42). Model E combines B, C, and D with weights 20%, 20%, and 60%. Model F is a separate character-only baseline. The later v3 pipeline reached 0.999551 AUROC in a holdout run and its full data Kaggle public score was 0.98552

Q1.3 [4 points]. For at least **two** approaches, which hyperparameters and candidate values did you try? How did you select the final settings without using Kaggle test labels?

For the logistic and SVM models C sets regularization strength; larger values apply a weaker penalty. In B, γ sets the weight of the style features. In D, we varied tree size, the minimum number of examples per leaf, and learning rate. The table shows the values tried and the setting chosen for each model.

Model	Parameters/values searched	Chosen setting
B: logistic	$C \in \{0.5, 1, 2, 3, 5, 8\}$; style scaling $\gamma \in \{0.10, 0.15, 0.25\}$	$C = 5$, $\gamma = 0.15$ (inner CV 0.9694)
C: linear SVM	$C \in \{0.2, 0.5, 1.0, 1.5, 2.0\}$; sigmoid calibration	$C = 1.0$ (inner CV 0.9727)
D: LightGBM	Leaves $\in \{15, 31, 63\}$; min. children $\in \{30, 50\}$; rate $\in \{0.04, 0.05\}$	Leaves 31, min. 50, rate 0.05 (inner CV 0.9764)
F: char logistic	Character $(3, 5)$ fixed; $C \in \{0.1, 1, 10\}$	$C = 10$

For models B, C, and D, we compared the settings above with five-fold cross-validation inside the 8000 training rows (seed 42). The separate 2000 row validation set was reserved for the model comparison; hidden test labels were never used.

For Model F, we used those same validation IDs to compare C values. Its validation AUROCs were 0.933322, 0.961446, and 0.968976 for C values (0.1, 1, 10) respectively so $C = 10$ was selected. The linear searches adjusted how strongly complexity was penalized; the tree search controlled model size and learning speed to limit overfitting.

Q1.4 [3 points]. Which approach did you choose for your final submission? Explain using validation performance, computational cost and/or stability. If you later changed the model, explain why.

We first chose Model E because its 0.9837 validation AUROC beat the word only baseline's 0.9367, and it trained in about 11 seconds on CPU. It combined word and character pattern models with a model based on writing style. It also kept 0.9804 AUROC when formatting was removed, whereas a layout dependent tree model fell to 0.8109. An early Kaggle submission scored 0.9395 publicly suggesting the fixed validation split missed some test differences.

V2 added topic masked patterns and a more stable blend, reaching 0.97446. We then submitted v3, which blends nine classifiers and adds examples given confident provisional labels by its first stage; it scored 0.98552 publicly. V3 is the team's intended final submission model. Its 0.999551 local holdout AUROC came from a related 8000/2000 validation run, not the exact full data submitted fit.

4. Effect of training data size (10 points)

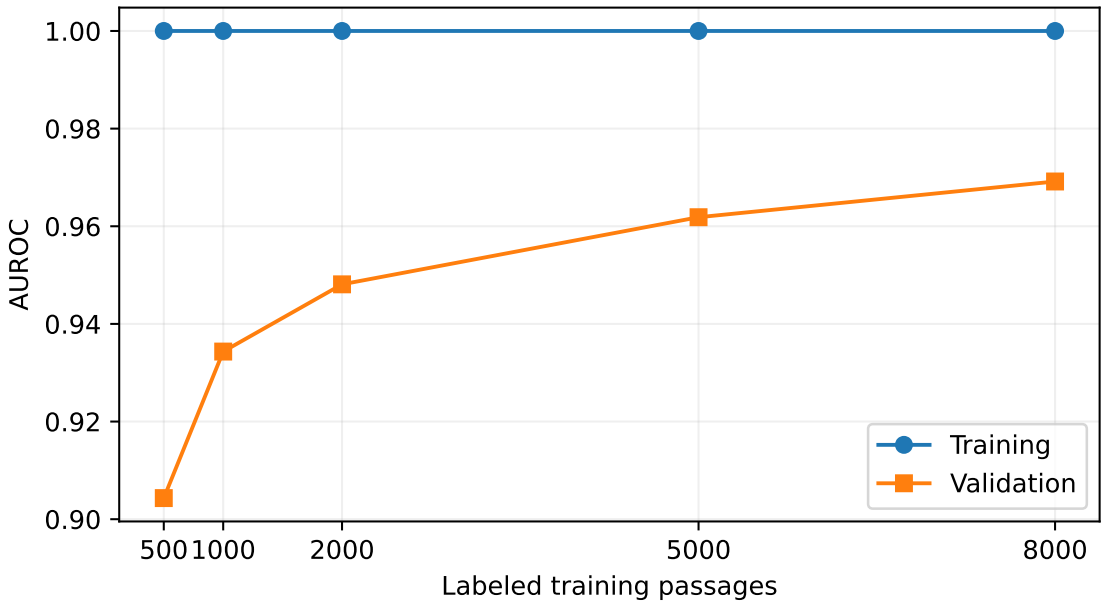
Learning goal: understand the effect of the number of labeled examples on fitting and generalization. This is an empirical exercise; no formal sample-complexity theory is required.

Q2.1 [4 points]. Choose one model configuration. Train it with 500, 1,000, 2,000, 5,000 and 8,000 examples sampled from your 8,000-row training partition. Keep the same validation set and hyperparameters.

We used the same character-pattern logistic model for all five runs and changed only how many labeled passages it learned from. We kept $C = 10$ and tested every run on the same 2000 validation passages. For each size, we sampled training passages with the same human/AI proportions (still using seed 42), then built a new character vocabulary and fitted a new model from that subset.

Training rows	Training AUROC	Validation AUROC	Notes (optional)
500	1.000000	0.904324	Character TF–IDF, $C = 10$
1,000	1.000000	0.934335	Same validation set
2,000	1.000000	0.947913	Same validation set
5,000	0.999998	0.962446	Same validation set
8,000	0.999897	0.968976	Same validation set

Q2.2 [3 points]. Insert a plot of training and validation AUROC versus training-set size.



Q2.3 [3 points]. What does the curve suggest about overfitting, underfitting, and the value of more labeled examples?

The model scores almost perfectly on the passages it learned from at every size, so it does not appear to be too simple for the training data. With only 500 passages, its validation AUROC is 0.9043; with 8000, it reaches 0.9690.

This large early gap between training and validation scores suggests overfitting: the model learns the small training sample better than it handles new passages. More labeled data helps, though the gains get smaller. The last 3000 passages raise validation AUROC by about 0.0065. Because we used one split and one sample at each size, we cannot predict how much another batch of labels would help.

5. Learning from unlabeled data (5 points)

Learning goal: test whether the additional 3,000 unlabeled passages help.

Q3.1 [2 points]. Describe one method that uses `unlabeled.csv` (e.g., self-training, confidence-based pseudo-labeling, clustering, or feature learning). How did you prevent validation-label leakage?

We trained the character only logistic model (Model F) on 8000 labeled passages, and then used it to estimate AI probabilities for the unlabeled passages. We removed duplicate texts and any exact matches to labeled texts. We kept only confident predictions: $p \leq 0.05$ became a provisional human label (540 passages), and $p \geq 0.95$ became a provisional AI label (572 passages).

We trained a new copy of Model F on the original 8000 passages plus these 1112 provisionally labeled passages. We did not use validation labels to select or label them; those labels were used only to measure the result. The final v3 model uses a different pseudo labeling setup, so this experiment does not measure v3's second stage.

Q3.2 [2 points]. Compare against a supervised baseline under the same held-out validation protocol.

Method	Validation AUROC
Supervised only (character baseline)	0.968976
With 1112 pseudo labeled rows	0.968447

Q3.3 [1 point]. Did it help? Give a concrete, evidence-based explanation (including possible noise in pseudo-labels).

Adding the 1112 provisionally labeled passages did not help Model F: validation AUROC fell from 0.968976 to 0.968447, a drop of 0.000529. A confident prediction can still be wrong, and retraining on wrong labels may repeat the first model's mistakes.

We do not know the true labels of those passages, so this is a possible explanation rather than a proven cause. We tried one data split and one pair of probability cutoffs; this result also does not tell us whether v3's different second stage helps.

6. Generalization and error analysis (20 points)

Learning goal: characterize how well the selected model generalizes and where it fails. Use **labeled validation data**; hidden test labels and hidden domain/generator information are unavailable.

For this analysis we split the 10000 labeled passages by class (seed 42): 8000 for training and 2000 held out for validation. Any v3 features that learn from labels, including the small language model and human reference genre statistics, used only the 8000 training rows.

In its second stage v3 assigned provisional labels to passages from `unlabeled.csv` and `test.csv` without seeing validation labels. This CPU run fitted each component once. The Kaggle submission instead combines five fits trained on all 10000 labeled rows. These results therefore test the v3 approach on held out passages, not the exact model submitted to Kaggle.

Q4.1 [5 points]. Compare training and validation AUROC for your final selected model. Is there a gap? What explanations are plausible?

The full two stage v3 run scored 1.000 AUROC on its training passages and 0.999551 on the held out passages, a small gap of 0.000449. That nearly perfect validation score may still be optimistic. A stage one version that ignored formatting scored 0.987841 on the same holdout.

In the labeled training data 66.0% of AI passages contained a double newline, compared with 0% of human passages; the model may therefore benefit from a formatting clue that does not hold elsewhere. The full data submission scored 0.98552 on Kaggle's public test subset, which also suggests that this random validation split was easier than the public test data. The public score does not show which individual passages were wrong, and the private score is still unknown.

Q4.2 [5 points]. Evaluate performance over at least three text-length bins (define the bins), or another observable subgroup available in your data. Explain why you chose the grouping.

We grouped validation passages by word count to see whether length affects performance. Each of the three groups includes several hundred passages and both human and AI writing.

Subgroup / bin	Count	Validation AUROC
75–165 words	578	0.999661
166–245 words	832	0.999147
246–350 words	590	0.999942

The three groups contain 260/318, 481/351, and 259/331 human/AI passages respectively. At a probability cutoff of 0.49, they have 4/2, 2/8, and 0/2 errors respectively (human passages called AI / AI passages called human). Eight of the twelve AI passages wrongly classified as human are in the middle group. With so few errors, we cannot conclude that "medium length" passages are generally harder.

Q4.3 [5 points]. Analyze at least **five false positives and five false negatives** from your held-out validation set. Provide example IDs, predicted probabilities, and short failure hypotheses. Do not paste full passages unless essential.

At the usual 0.50 probability cutoff, v3 wrongly flagged 4 human passages as AI (false positives) and missed 12 AI passages (false negatives). To examine at least five of each type, we lowered the cutoff to 0.49; this gives 6 false positives and 12 false negatives. The table shows five examples of each. The possible reasons come from a brief reading of the passages and are not proven causes.

ID	Error type	P(AI)	Possible reason
7767	FP	0.6644	Its short and "polished" bio may read like an AI summary
4062	FP	0.5646	The tidy and neat list of song facts may read like an AI summary as well
3660	FP	0.5634	Repeated names and line breaks may look computer generated
5079	FP	0.5619	The season recap repeats formal phrases which could look AI-written
3571	FP	0.4986	Repeated opinions and the word *movie* may look AI-written too
1362	FN	0.0535	Dates and names may make this AI bio seem human written
9298	FN	0.1482	Names and dated events may make this AI bio seem human written too
4519	FN	0.1503	Lowercase first person wording may look like a casual human post
1834	FN	0.2234	The timeline with dates may make this AI text seem like a human history article
5498	FN	0.2277	Amounts and line breaks may make this AI text look like a human written recipe

FP means human predicted AI; FN means AI predicted human. The threshold is 0.49.

Q4.4 [5 points]. What recurring failure patterns did you identify, and how might you address one of them? Distinguish observed patterns from speculation.

Among these 10 examples several human passages written like encyclopedia entries were labeled AI, while several AI biographies and one AI recipe were labeled human. An AI forum style passage was also missed, so genre alone does not explain every error.

We believe that ten examples are too few to tell us how common these problems are. We also saw evidence that formatting matters as well: the full first stage blend scored 0.999561 held out AUROC and its version without formatting features scored 0.987841.

The final v3 public Kaggle score was lower than its random holdout score. We would try reducing the weight of the formatting based component and test on different genres and passages with altered line breaks before changing the submission. That test could show whether the formatting clue remains useful on different data; we cannot yet say that this change would improve the private score.

7. Open-ended improvements and insights (15 points)

Learning goal: propose and test one improvement beyond the required baseline comparisons. This can be simple: a targeted feature change, alternate regularization scheme, ensemble, different representation, augmentation, or training approach. Sophistication alone does not earn credit.

Q5.1 [3 points]. What specific question or hypothesis motivated your improvement? What weakness or opportunity did you observe?

A later version of our nine-model ensemble made 20 mistakes on a 2,000-passage holdout set: it wrongly flagged six human passages as AI and missed 14 AI passages. This is a separate evaluation from the v3 run in Section 6.

Some missed AI passages began with Markdown quote marks, while some human lists of names looked repetitive. We also expected poetry to be difficult because its short lines and rhyme differ from ordinary prose. Our question was simple: Could targeted checks for these patterns improve the ensemble's predictions?

Q5.2 [4 points]. What exactly did you change? Describe the baseline, the new approach and what you held fixed for a fair comparison.

We started with the same ensemble and its existing probabilities. Then we tried three specific ideas: remove leading Markdown quote marks from lines, measure whether repeated phrases contain capitalized names, and measure line length and rough rhyme patterns in verse.

For the exported test predictions, we raised some scores for quote-prefixed technical passages and adjusted some verse scores using the rhyme and line-length checks. We did not retrain the ensemble with these new features. The holdout demonstration also changed scores for specific IDs already known to be errors, so it is useful for exploring the idea but is not a fair test on unseen passages.

Q5.3 [4 points]. What were the measured results?

Step	Holdout AUROC	Errors at 0.50	Kaggle public AUROC
Original ensemble	0.999449	20	0.98629
Edits to known holdout errors	0.999506	12	—
Exported test candidate	—	—	0.98640

The holdout numbers come from the notebook's executed output. The public scores are recorded in the team's model results log. The holdout change is a post hoc diagnostic because the edited IDs were chosen after their errors were known; the Kaggle score measures a separate test-file candidate.

Q5.4 [4 points]. What did you learn? Did the results support the hypothesis? What limits the conclusion, and what would you try next?

The error-directed candidate improved the recorded public score from 0.98629 to 0.98640, a gain of 0.00011. That suggests the targeted checks may help, but it does not tell us which check mattered. We cannot treat the drop from 20 to 12 holdout errors as evidence of generalization because those predictions were adjusted after we inspected the mistakes.

The public leaderboard also covers only part of the test data. Next, we would define the rules without looking at the evaluation IDs, apply them to every passage in a fresh holdout or cross-validation fold, and test quote-prefixed text and verse separately before relying on the change.

8. Code submission, reproducibility and AI assistance (10 points)

Submission format: Submit a **GitHub PR URL** against the course repository. We will inspect the final code and its development history, not just the final snapshot. Do not squash away meaningful intermediate commits before review. Template here.

Q6.1 [5 points]. Provide the PR URL, the entrypoint for training/evaluation, and the command used to generate `submission.csv`. State dependencies and random seeds.

Our current fork review PR is PR #10. The required course-repository PR URL is pending the team's final submission. To reproduce the final CSV, use Python 3.12 on CPU and place `train.csv`, `unlabeled.csv`, `test.csv`, and `sample_submission.csv` in the repository's `data/` directory. From the repository root, run:

```
python -m pip install -r submissions/team-35/requirements.txt \
  nbconvert ipykernel
cd submissions/team-35
jupyter nbconvert --to notebook --execute --inplace \
  --ExecutePreprocessor.timeout=-1 final_model_improvements.ipynb
jupyter nbconvert --to notebook --execute --inplace \
  --ExecutePreprocessor.timeout=-1 error_directed_model_refinement.ipynb
cp submission_error_directed_refinement.csv submission.csv
```

The first notebook creates the base predictions and artifacts needed by the second. The last command names the refined output `submission.csv`; the first notebook's own CSV is an earlier model. The configuration uses seed 42 for the main split and seed 43 for a secondary check. Competition data and generated CSVs are kept out of Git.

Q6.2 [3 points]. Give two or three examples of changes you made during development. Link to relevant commits or PR discussion and briefly explain what the experiments taught you.

- `5e8256c`: We measured a character-pattern baseline at several training sizes. More labeled passages improved validation AUROC, while adding confident pseudo-labels did not help in that separate test.
- `8930188`: More model views and leakage checks raised public AUROC from v3's 0.98552 to 0.98629, though the test gap remained.
- `c7f2ca6`: Quote and verse adjustments reached 0.98640 publicly. The ID-edited holdout comparison was only diagnostic.

Q6.3 [2 points]. How did you use AI tools (such as Claude, ChatGPT or Copilot) for code, debugging, analysis or writing? What did you verify or modify yourself?

We used AI assistants, including ChatGPT/Codex, to brainstorm approaches, draft code, suggest improvements, and help format and organize files and report text. The team chose the direction of the experiments, reviewed and changed suggested code and wording, checked the reasoning, and validated the reported results. We compared numerical claims with saved notebook outputs and Kaggle score records rather than treating AI suggestions as measurements.

AI policy. You may use AI assistants for coding, debugging, explanations, brainstorming, and editing. We expect each team to exercise its own judgment, run experiments, check outputs, and understand its submitted approach. Sending the entire assignment to an AI tool and submitting its unverified output without meaningful involvement does not satisfy the objectives of this project. We will use the PR history, reproducible experiments and report together as evidence of engagement. *Commit counts and the presence of AI-generated code are not themselves grading criteria.* Do not fabricate results or retrieve hidden test labels.

Final submission checklist

☐ Final probabilities submitted to Kaggle for every `test.csv` ID.

☐ Completed report, with all required answers, tables, figures and sources, submitted as PDF.

☐ GitHub PR linked on the report title page and in the course submission portal.

☐ Code and dependencies recreate final predictions (within reasonable numerical tolerance).

☐ Relevant experiments and seed/configuration details are recorded.

☐ AI-use disclosure is complete; hidden labels were not retrieved externally.